



RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

LABORATORY OBSERVATION

Submitted by

SRINIVASAN S 231701054

in partial fulfillment for the course

**AI23331- FUNDAMENTALS OF
MACHINE LEARNING**

Academic Year 2025–2026

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



AI23331

FUNDAMENTALS OF MACHINE LEARNING

Laboratory Observation Note Book

Name :

Year / Branch / Section :

Register No. :

Semester :

Academic Year :



RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: **SANJEEVE R V**

Academic Year: **III** Semester: **VI** Branch: **CSD**

Register No.

231701048

*Certified that this is the bonafide record of work done by the above student in
the.....Laboratory
during the academic year 2025- 2026*

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner

External Examiner

INDEX

Reg. No. : 231701048 Name : R V SANJEEVE

Year : III Branch : CSD Sec : A

S.No.	Date	Title	Page No.	Teacher's Signature / Remarks
1		A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESION	02	
2		A PYTHON PROGRAM TO IMPLEMENT SIMPLE LINEAR REGRESSION	06	
3		A PYTHON PROGRAM TO IMPLEMENT LOGISTIC MODEL	11	
4		A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON	17	
5		A PYTHON PROGRAM TO IMPLEMENT MULTI LAYER PERCEPTRON WITH BACKPROPAGATION	21	
6		FACE RECOGNITION USING SVM CLASSIFIER	24	
7		A PYTHON PROGRAM TO IMPLEMENT DECISION TREE CLASSIFIER	28	
8		A PYTHON PROGRAM TO IMPLEMENT ADA BOOSTING	31	
9		A PYTHON PROGRAM TO IMPLEMENT K-NEAREST NEIGHBORS (KNN) & K-MEAN CLASSIFIER	36	
10		A PYTHON PROGRAM TO	44	

		IMPLEMENT DIMENSIONALITY REDUCTION USING PCA		
--	--	---	--	--

Ex. No. : 01 **Date:**

Register No.: 231701054 **Name:** Srinivasan S

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESION

Aim:

Develop a Python program to implement **univariate, bivariate, and multivariate regression** models and analyze the relationship between dependent and independent variables.

Requirements: Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
df=pd.read_csv("iris.csv")
df.head(150)
df.shape
```

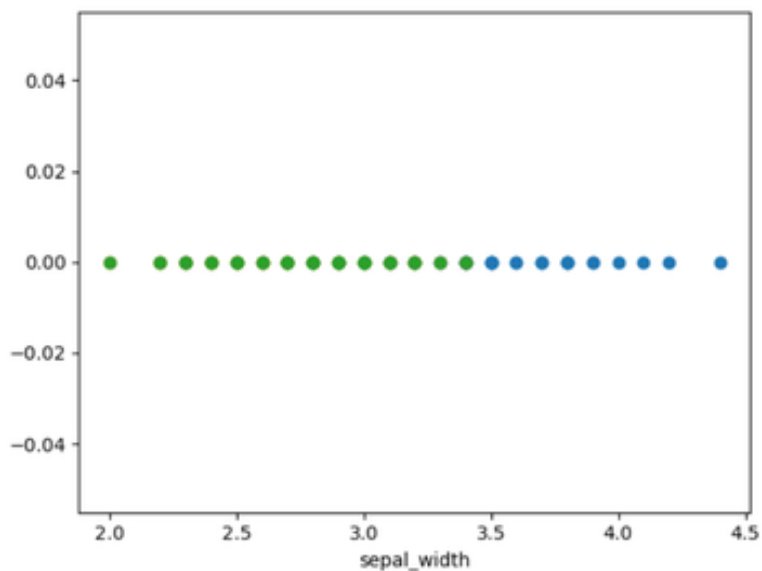
#univariate for sepal width

```

df.loc[df['species']=='setosa']
df_Setosa=df.loc[df['species']=='setosa']
df_Virginica=df.loc[df['species']=='versicolor']
df_Versicolor=df.loc[df['species']=='versicolor']
plt.scatter(df_Setosa['sepal_width'], np.zeros_like(df_Setosa['sepal_width']))
plt.scatter(df_Virginica['sepal_width'], np.zeros_like(df_Virginica['sepal_width']))
plt.scatter(df_Versicolor['sepal_width'], np.zeros_like(df_Versicolor['sepal_width']))
plt.xlabel('sepal_width')
plt.show()

```

#Output:



#univariate for petal length

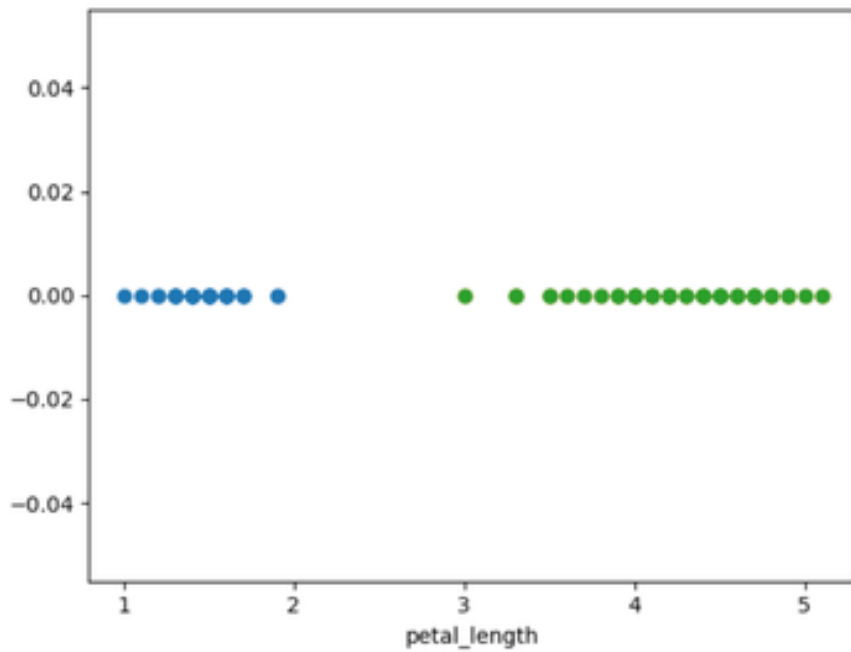
```

df.loc[df['species']=='setosa']
df_Setosa=df.loc[df['species']=='setosa']
df_Virginica=df.loc[df['species']=='versicolor']
df_Versicolor=df.loc[df['species']=='versicolor']
plt.scatter(df_Setosa['petal_length'], np.zeros_like(df_Setosa['petal_length']))

```

```
plt.scatter(df_Virginica['petal_length'],np.zeros_like(df_Virginica['petal_length']))  
plt.scatter(df_Versicolor['petal_length'],np.zeros_like(df_Versicolor['petal_length']))  
plt.xlabel('petal_length')  
plt.show()
```

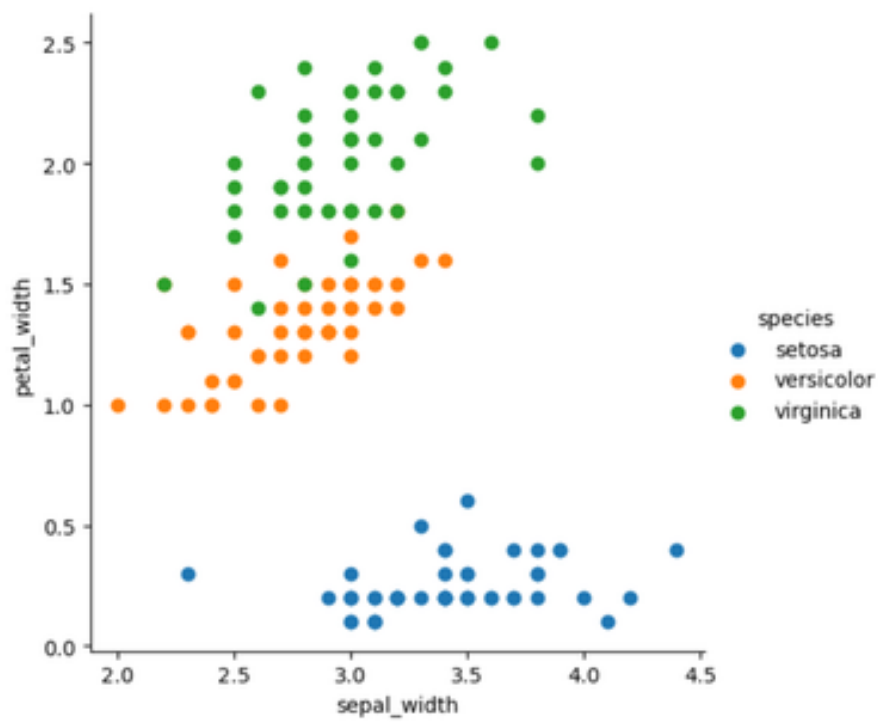
#Output:



#bivariate sepal.width vs petal.width

```
sns.FacetGrid(df,hue='species',height=5).map(plt.scatter,"sepal_width","petal_width").add_legend();
```

#Output:

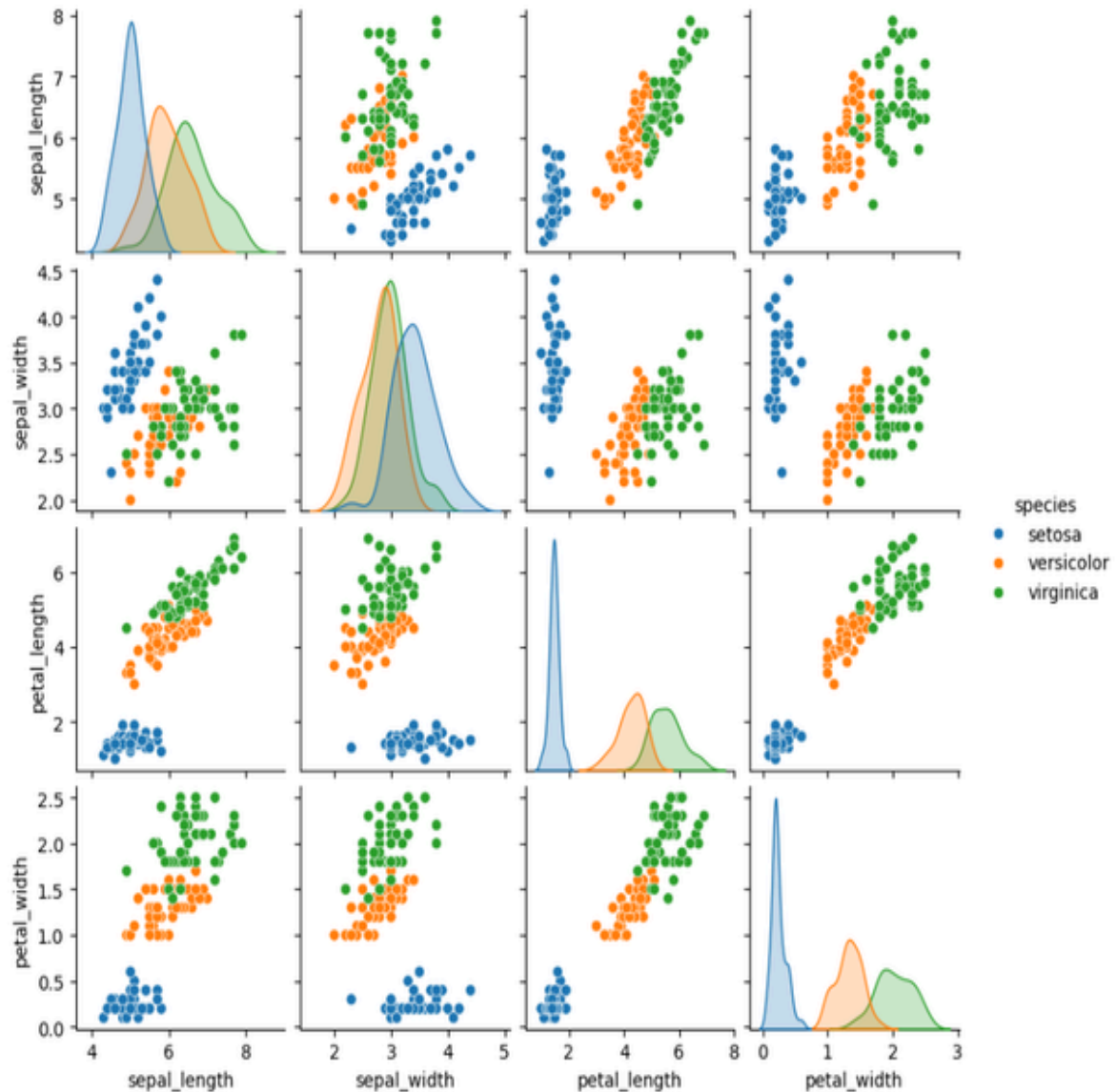


#multivariate all the features

```
sns.pairplot(df,hue="species",height=2)
```

#Output:

```
[19]: <seaborn.axisgrid.PairGrid at 0x276d0abb380>
```



Result:

the Python program for univariate, bivariate, and multivariate regression was successfully implemented using the Iris.csv dataset in Jupyter Notebook, and the regression models were able to analyze and predict relationships between variables accurately.

Ex. No. : 02Date:

Register No.: 231701054 Name: Srinivasan S

A PYTHON PROGRAM TO IMPLEMENT SIMPLE LINEAR REGRESSION

Aim:

To write a python program to implement Simple linear regression using Least Square Method

Requirements: Jupyter Notebook with Python, dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("dia.csv")

print("First 5 Rows:")
print(data.head())
```

First 5 Rows:

	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	\
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	
1	0.0	0.0	0.0	0.0	25.0	1.0	0.0	
2	0.0	1.0	1.0	1.0	28.0	0.0	0.0	
3	0.0	1.0	0.0	1.0	27.0	0.0	0.0	
4	0.0	1.0	1.0	1.0	24.0	0.0	0.0	

	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	\
0	0.0	0.0	0.0	...	1.0	
1	0.0	1.0	0.0	...	0.0	
2	0.0	0.0	1.0	...	1.0	
3	0.0	1.0	1.0	...	1.0	
4	0.0	1.0	1.0	...	1.0	

	NoDocbcCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	\
0	0.0	5.0	18.0	15.0	1.0	0.0	9.0	4.0	
1	1.0	3.0	0.0	0.0	0.0	0.0	7.0	6.0	
2	1.0	5.0	30.0	30.0	1.0	0.0	9.0	4.0	
3	0.0	2.0	0.0	0.0	0.0	0.0	11.0	3.0	
4	0.0	2.0	3.0	0.0	0.0	0.0	11.0	5.0	

	Income
0	3.0
1	1.0
2	8.0
3	6.0
4	4.0

[5 rows x 22 columns]

```
data = data[['BMI', 'Diabetes_012']].dropna()
```

```
X = data[['BMI']]
```

```
y = data['Diabetes_012']
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
print("Training size:", X_train.shape)
```

```
print("Testing size:", X_test.shape)
```

```
Training size: (202944, 1)
```

```
Testing size: (50736, 1)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
intercept = model.intercept_
```

```
slope = model.coef_[0]
```

```
print("Intercept (b0):", intercept)
```

```
print("Slope (b1):", slope)
```

```
Intercept (b0): -0.37741687481513303
```

```
Slope (b1): 0.023789834877730138
```

```
print("\nRegression Equation:")
```

```
print(f"Diabetes Risk = {intercept:.2f} + ({slope:.2f} × BMI)")
```

```
Regression Equation:
```

```
Diabetes Risk = -0.38 + (0.02 × BMI)
```

```
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
```

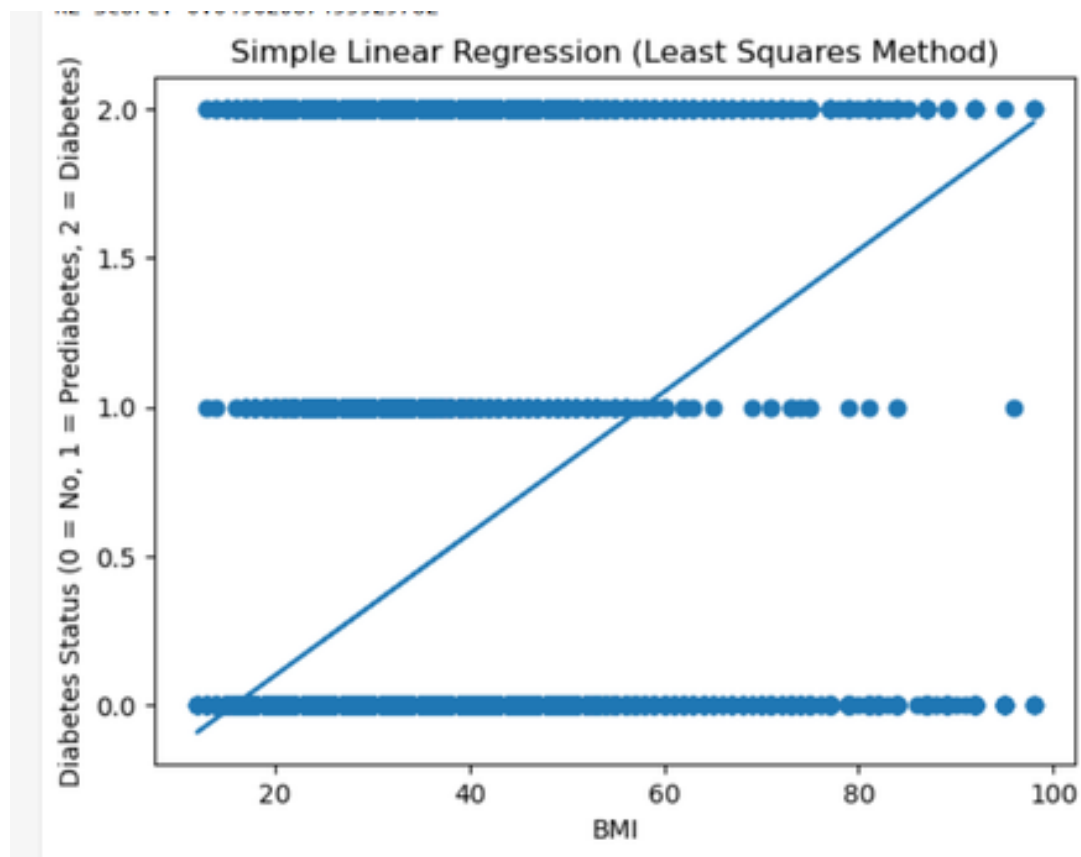
```
r2 = r2_score(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
```

```
print("R2 Score:", r2)
```

```
Mean Squared Error: 0.4598547822512952  
R2 Score: 0.04902087455929782
```

```
plt.figure()  
plt.scatter(X, y)  
plt.plot(X, model.predict(X))  
plt.xlabel("BMI")  
plt.ylabel("Diabetes Status (0 = No, 1 = Prediabetes, 2 = Diabetes)")  
plt.title("Simple Linear Regression (Least Squares Method)")  
plt.show()
```



```
bmi = np.array([[30]])  
predicted_risk = model.predict(bmi)
```

```
print("Predicted Diabetes Risk for BMI 30:", predicted_risk[0])
```

```
Predicted Diabetes Risk for BMI 30: 0.33627817151677114
```

Result:

The Python program for **Simple Linear Regression using the Least Squares Method** was successfully implemented using the **dataset** in Jupyter Notebook. The model analyzed the relationship between the independent and dependent variables and predicted values using the best-fit regression line obtained through the least squares technique.

Ex. No. : 03 Date:

Register No.: 231701054 Name: Srinivasan S

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC MODEL

Aim:

To write a python program to implement Logistic Model

Requirements: Jupyter Notebook with Python, dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load Dataset

```
data = pd.read_csv("dia.csv")
data.head()
```

	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	NoDocbcCost	GenHlth	Ment
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	0.0	0.0	0.0	...	1.0	0.0	5.0	
1	0.0	0.0	0.0	0.0	25.0	1.0	0.0	0.0	1.0	0.0	...	0.0	1.0	3.0	
2	0.0	1.0	1.0	1.0	28.0	0.0	0.0	0.0	0.0	1.0	...	1.0	1.0	5.0	
3	0.0	1.0	0.0	1.0	27.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	
4	0.0	1.0	1.0	1.0	24.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	

5 rows × 22 columns

Step 3: Dataset Information

data.info()

data.describe()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 253680 entries, 0 to 253679
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Diabetes_012                          253680 non-null float64
1   HighBP                                253680 non-null float64
2   HighChol                              253680 non-null float64
3   CholCheck                             253680 non-null float64
4   BMI                                    253680 non-null float64
5   Smoker                                253680 non-null float64
6   Stroke                                253680 non-null float64
7   HeartDiseaseorAttack                  253680 non-null float64
8   PhysActivity                           253680 non-null float64
9   Fruits                                 253680 non-null float64
10  Veggies                                253680 non-null float64
11  HvyAlcoholConsump                      253680 non-null float64
12  AnyHealthcare                           253680 non-null float64
13  NoDocbcCost                             253680 non-null float64
14  GenHlth                                 253680 non-null float64
15  MentHlth                                253680 non-null float64
16  PhysHlth                                253680 non-null float64
17  DiffWalk                                253680 non-null float64
18  Sex                                      253680 non-null float64
19  Age                                      253680 non-null float64
20  Education                               253680 non-null float64
21  Income                                  253680 non-null float64
dtypes: float64(22)
memory usage: 42.6 MB
```

[3]:	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruit
count	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000
mean	0.296921	0.429001	0.424121	0.962670	28.382364	0.443169	0.040571	0.094186	0.756544	0.63425
std	0.698160	0.494934	0.494210	0.189571	6.608694	0.496761	0.197294	0.292087	0.429169	0.48163
min	0.000000	0.000000	0.000000	0.000000	12.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	1.000000	24.000000	0.000000	0.000000	0.000000	1.000000	0.000000
50%	0.000000	0.000000	0.000000	1.000000	27.000000	0.000000	0.000000	0.000000	1.000000	1.000000
75%	0.000000	1.000000	1.000000	1.000000	31.000000	1.000000	0.000000	0.000000	1.000000	1.000000
max	2.000000	1.000000	1.000000	1.000000	98.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Step 4: Define Features and Target

```
X = data.drop("Diabetes_012", axis=1)
y = data["Diabetes_012"]
```

Step 5: Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 6: Train Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

LogisticRegression		
Parameters		
penalty		'l2'
dual		False
tol		0.0001
C		1.0
fit_intercept		True
intercept_scaling		1
class_weight		None
random_state		None
solver		'lbfgs'
max_iter		1000
multi_class		'deprecated'
verbose		0
warm_start		False
n_jobs		None
l1_ratio		None

Step 7: Predict Results

```
y_pred = model.predict(X_test)
```

Step 8: Model Evaluation Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Accuracy: 0.8482339955849889

Confusion Matrix

```
print("Confusion Matrix:")  
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:  
[[41753    0  1042]  
 [   871    0    73]  
 [  5714    0  1283]]
```

Classification Report

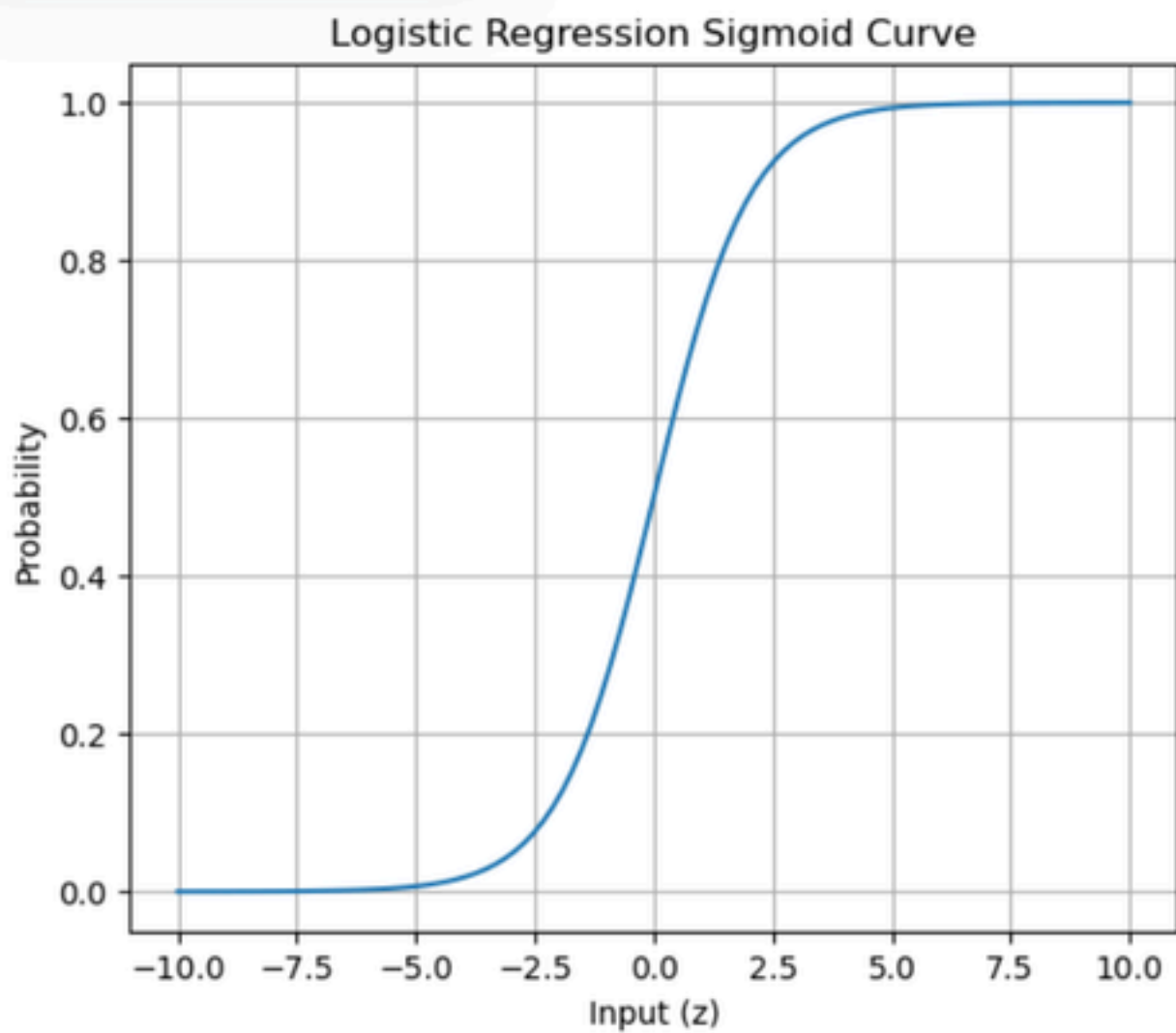
```
print("Classification Report:")  
print(classification_report(y_test, y_pred))
```

```
Classification Report:  
              precision    recall  f1-score   support  
  
    0.0         0.86      0.98      0.92     42795  
    1.0         0.00      0.00      0.00        944  
    2.0         0.54      0.18      0.27     6997  
  
   accuracy              0.85     50736  
  macro avg              0.47      0.39      0.40     50736  
weighted avg              0.80      0.85      0.81     50736
```

```
curve = np.linspace(-10, 10, 100)
```

```
y = 1 / (1 + np.exp(-x))
```

```
plt.figure(figsize=(6,5))  
  
plt.plot(x, y)  
  
plt.title("Logistic Regression Sigmoid Curve")  
  
plt.xlabel("Input (z)")  
  
plt.ylabel("Probability")  
  
plt.grid(True)  
  
plt.show()
```



Result:

Thus, the Logistic Regression model was successfully implemented using Python and evaluated using the **dia.csv dataset**. The model analyzed the relationship between the input health factors and the diabetes outcome, and its performance was evaluated using accuracy, confusion matrix, and classification report.

Ex. No. : 04 Date:

Register No.: 231701054 Name: Srinivasan S

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON

Aim:

To write a python program to implement Single Layer perceptron

Requirements: Jupyter Notebook with Python, dataset, and required Python libraries.

PROGRAM:**Step 1: Import Libraries**

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import Perceptron
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Dataset

```
data = pd.read_csv("dia.csv")
```

```
data.head()
```

	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	NoDocbcCost	GenHlth	Ment
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	0.0	0.0	0.0	...	1.0	0.0	5.0	
1	0.0	0.0	0.0	0.0	25.0	1.0	0.0	0.0	1.0	0.0	...	0.0	1.0	3.0	
2	0.0	1.0	1.0	1.0	28.0	0.0	0.0	0.0	0.0	1.0	...	1.0	1.0	5.0	
3	0.0	1.0	0.0	1.0	27.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	
4	0.0	1.0	1.0	1.0	24.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	

5 rows × 22 columns

Step 3: Define Features and Target

```
X = data.drop("Diabetes_012", axis=1)
```

```
y = data["Diabetes_012"]
```

Step 4: Feature Scaling

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 5: Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

Step 6: Train Perceptron Model

```
model = Perceptron()
```

```
model.fit(X_train, y_train)
```

[7]:

Perceptron		
Parameters		
	penalty	None
	alpha	0.0001
	l1_ratio	0.15
	fit_intercept	True
	max_iter	1000
	tol	0.001
	shuffle	True
	verbose	0
	eta0	1.0
	n_jobs	None
	random_state	0
	early_stopping	False
	validation_fraction	0.1
	n_iter_no_change	5
	class_weight	None
	warm_start	False

Step 7: Predict Results

```
y_pred = model.predict(X_test)
```

Step 8: Model Evaluation Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.7877444023967203

Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[37204  103  5488]
 [  679    7   258]
 [ 4197   44 2756]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

    0.0         0.88        0.87        0.88        42795
    1.0         0.05        0.01        0.01          944
    2.0         0.32        0.39        0.36         6997

 accuracy                   0.79        50736
 macro avg              0.42        0.42        0.42        50736
 weighted avg           0.79        0.79        0.79        50736
```

Result:

Thus, the **Single Layer Perceptron model** was successfully implemented using Python and evaluated using the **dia.csv dataset**. The model learned the relationship between the input

health features and the diabetes outcome, and its performance was evaluated using accuracy, confusion matrix, and classification report.

Ex. No. : 05**Date:**

Register No.: 231701054 **Name: Srinivasan S**

A PYTHON PROGRAM TO IMPLEMENT MULTI LAYER PERCEPTRON WITH BACKPROPAGATION

Aim:

To implement a Multi-Layer Perceptron (MLP) using Backpropagation in Python for predicting diabetes using the diabetes dataset.

Requirements:

- Jupyter Notebook or Google Colab
- Python
- diabetes.csv dataset
- Required Python libraries

Algorithm:

Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron is a type of Artificial Neural Network used for classification problems. It contains an input layer, hidden layers, and an output layer. The model learns patterns from data using the Backpropagation algorithm.

Backpropagation

Backpropagation is a learning algorithm used to reduce prediction error. The input data is passed through the network, error is calculated, and weights are updated automatically to improve accuracy.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

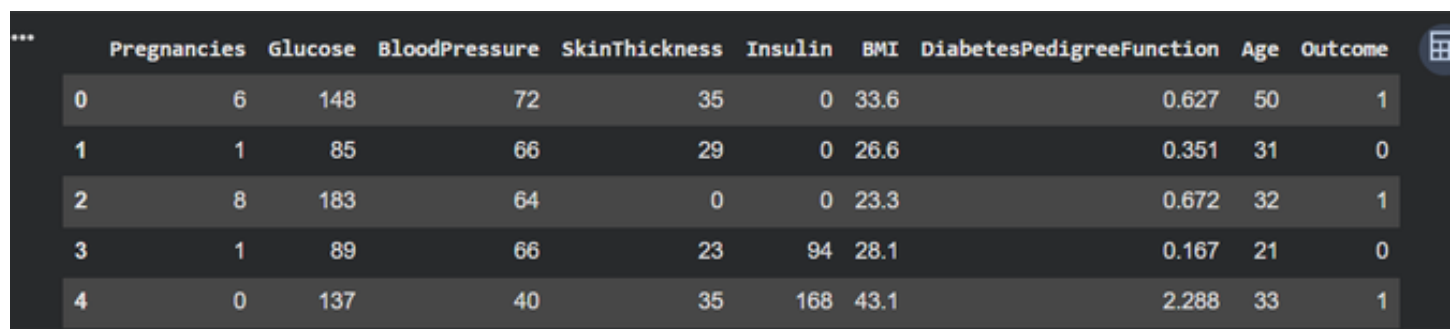
from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

Step 2: Load the Dataset

```
data = pd.read_csv("diabetes.csv")data.head()
```



The screenshot shows a Jupyter Notebook interface with a code cell containing `data.head()` and its output. The output is a table with 11 columns: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, and Outcome. The first five rows are displayed, showing various patient data points.

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Step 3: Split Dataset

```
X = data.drop("Outcome", axis=1)y = data["Outcome"]X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

Step 4: Feature Scaling

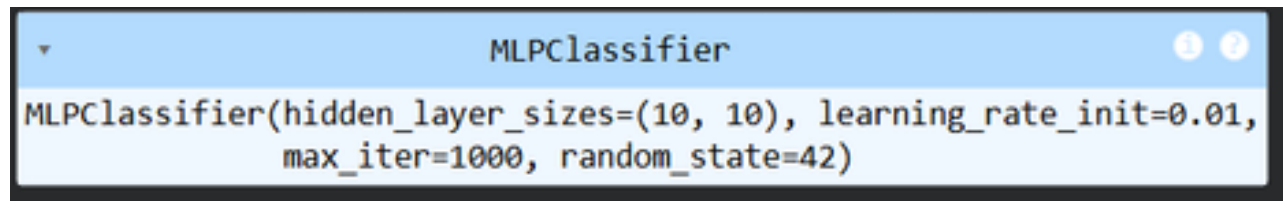
```
scaler = StandardScaler()X_train = scaler.fit_transform(X_train)X_test = scaler.transform(X_test)
```

Step 5: Initialize MLP Model

```
model = MLPClassifier(hidden_layer_sizes=(10, 10),max_iter=1000,learning_rate_init=0.01,random_state=42)
```

Step 6: Train Model

```
model.fit(X_train, y_train)
```



```
MLPClassifier  
MLPClassifier(hidden_layer_sizes=(10, 10), learning_rate_init=0.01,  
              max_iter=1000, random_state=42)
```

Step 7: Predict Results

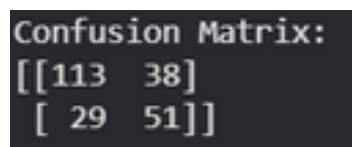
```
y_pred = model.predict(X_test)
```

Step 8: Model Evaluation

```
print("Accuracy Score:")print(accuracy_score(y_test, y_pred))
```

Confusion Matrix

```
print("Confusion Matrix:")print(confusion_matrix(y_test, y_pred))
```



```
Confusion Matrix:  
[[113  38]  
 [ 29  51]]
```

Classification Report

```
print("Classification Report:")print(classification_report(y_test, y_pred))
```

Classification Report:					
		precision	recall	f1-score	support
	0	0.80	0.75	0.77	151
	1	0.57	0.64	0.60	80
	accuracy			0.71	231
	macro avg	0.68	0.69	0.69	231
	weighted avg	0.72	0.71	0.71	231

Result:

Thus, the Multi-Layer Perceptron model was successfully implemented using backpropagation on the netfli_titles dataset and achieved high classification accuracy

Ex. No. : 06**Date:**

Register No.: 231701054 **Name: Srinivasan S**

FACE RECOGNITION USING SVM CLASSIFIER

Aim:

To develop a face recognition system using a Support Vector Machine (SVM) classifier by training on a folder-based image dataset and predicting the identity of a user-selected test image

Requirements:

- Jupyter Notebook / Google Colab
- Python
- Face image dataset (folder-based structure)
- Libraries: numpy, opencv, sklearn

Algorithm:

Support Vector Machine (SVM)

SVM is a supervised machine learning algorithm used for classification. It finds the optimal hyperplane that separates different classes with maximum margin.

Face Recognition Process:

1. Load dataset images
2. Convert images to grayscale
3. Resize and flatten images
4. Encode labels
5. Train SVM classifier
6. Predict test image class

PROGRAM:

Import Required Libraries

```
import os
import cv2
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Load Dataset

```
dataset_path = "/content/cricket_dataset/images"
```

Load Images and Labels

```
faces = []
```

```
labels = []
```

```
for person_name in os.listdir(dataset_path):
```

```
    person_folder = os.path.join(dataset_path, person_name)
```

```
    if not os.path.isdir(person_folder):
```

```
        continue
```

```
    for img_name in os.listdir(person_folder):
```

```
        img_path = os.path.join(person_folder, img_name)
```

```
        img = cv2.imread(img_path)
```

```
if img is None:
```

```
    continue
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
gray = cv2.resize(gray, (80, 80))
```

```
faces.append(gray.flatten())
```

```
labels.append(person_name)
```

```
faces = np.array(faces)
```

```
labels = np.array(labels)
```

```
print("Total Images Loaded:", len(faces))
```

Encode Labels

```
le = LabelEncoder() labels_encoded = le.fit_transform(labels)
```

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split( faces, labels_encoded, test_size=0.3,  
random_state=42)
```

Feature Scaling

```
scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test)
```

Initialize SVM Model

```
model = SVC(kernel='linear', probability=True, random_state=42)
```

Train Model

```
model.fit(X_train, y_train)
```

Prediction

```
y_pred = model.predict(X_test)
```

Predict New Image

```
def predict_player(image_path):    img = cv2.imread(image_path)    if img is None:        return  
"Image not found"    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)    gray = cv2.resize(gray,
```

```
(80, 80)) sample = gray.flatten().reshape(1, -1) sample = scaler.transform(sample) pred = model.predict(sample) return le.inverse_transform(pred)[0]
```

Test Prediction

```
print(predict_player("/content/test_image.jpg"))
```

OUTPUT



Result:

Thus, a Face Recognition system using Support Vector Machine (SVM) classifier was successfully implemented and used to predict the identity of a person from input images.

Ex. No. : 07**Date:**

Register No.: 231701054 **Name: Srinivasan S**

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE CLASSIFIER

Aim:

To implement the Decision Tree algorithm on the diabetes dataset and predict whether a person has diabetes based on medical attributes.

Requirements:

- Jupyter Notebook / Google Colab
- Python
- diabetes.csv dataset
- Libraries: pandas, numpy, sklearn, matplotlib

Algorithm:

Decision Tree Classifier

A Decision Tree is a supervised learning algorithm used for classification. It splits data into branches based on feature conditions and reaches a decision at leaf nodes.

Process:

1. Load dataset
2. Split features and target
3. Train Decision Tree model
4. Predict output
5. Evaluate model
6. Visualize tree

PROGRAM:

Upload Dataset

```
from google.colab import files  
uploaded = files.upload()
```

Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("diabetes.csv")
print("Dataset Preview:")
print(df.head())
```

Split Features and Target

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Train Decision Tree Model

```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
```

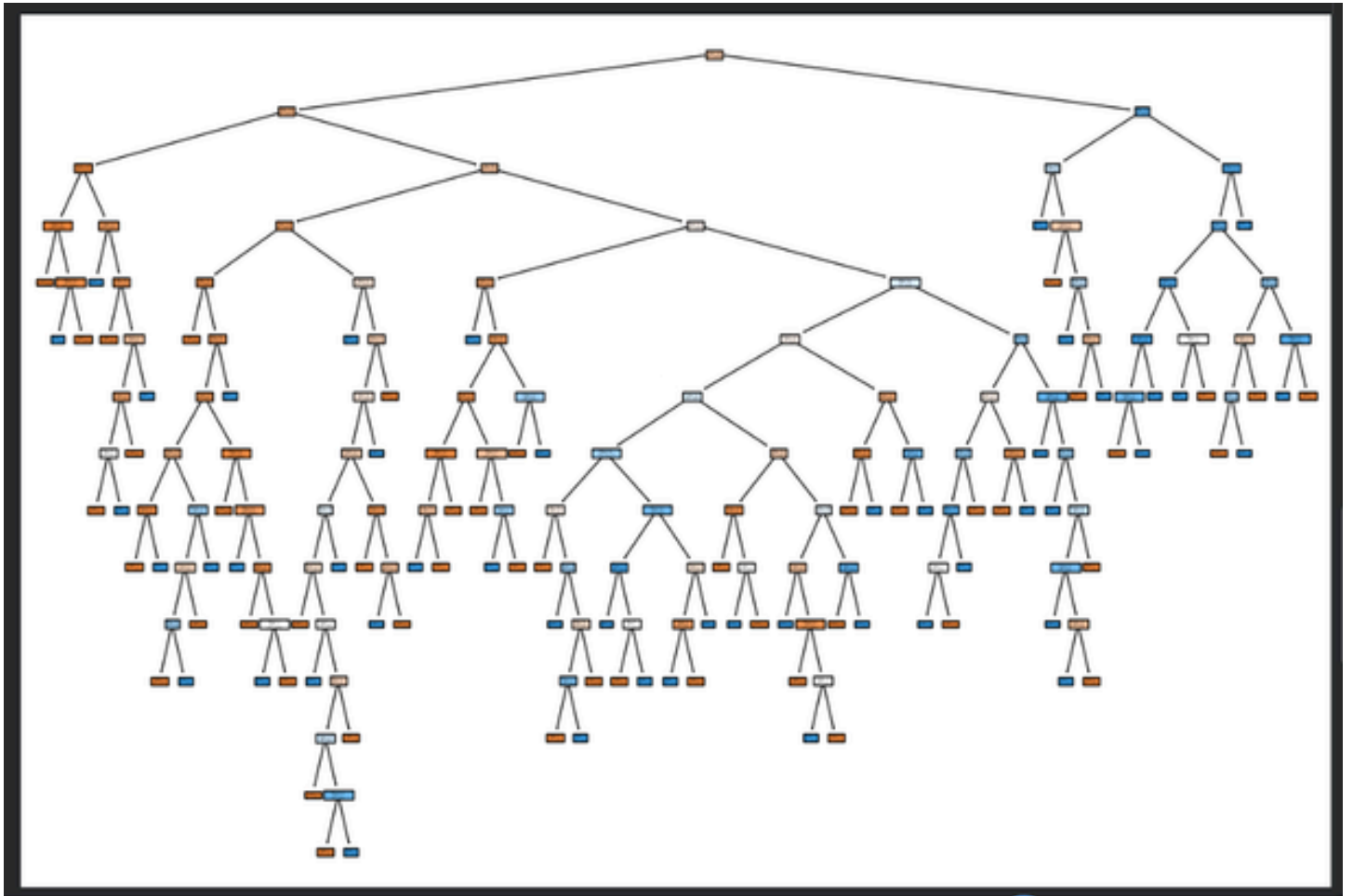
Prediction

```
y_pred = model.predict(X_test)
```

Decision Tree Visualization

```
plt.figure(figsize=(15,10))
tree.plot_tree(model, feature_names=X.columns, class_names=["No Diabetes", "Diabetes"], filled=True)
plt.show()
```

Output:



Result:

Thus, the Decision Tree classifier was successfully implemented using the diabetes dataset and used to predict whether a person has diabetes based on medical attributes.

Ex. No. : 08 Date:

Register No.: 231701054 Name: Srinivasan S

A PYTHON PROGRAM TO IMPLEMENT ADA BOOSTING

Aim:

To implement the AdaBoost boosting algorithm using the diabetes dataset and predict whether a person has diabetes based on medical attributes.

Requirements:

- Jupyter Notebook / Google Colab
- Python
- diabetes.csv dataset
- Libraries: pandas, numpy, sklearn, matplotlib, seaborn

Algorithm:

AdaBoost (Adaptive Boosting)

AdaBoost is an ensemble learning technique that combines multiple weak classifiers to form a strong classifier. It assigns higher weight to misclassified samples and improves accuracy iteratively.

Process:

1. Load dataset
2. Encode/prepare data
3. Split dataset
4. Train base model (Decision Stump)
5. Apply AdaBoost classifier
6. Predict output
7. Evaluate model

PROGRAM:

Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("diabetes.csv")
print("Dataset Preview:")
print(df.head())
```

Split Features and Target

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

Train-Test Split

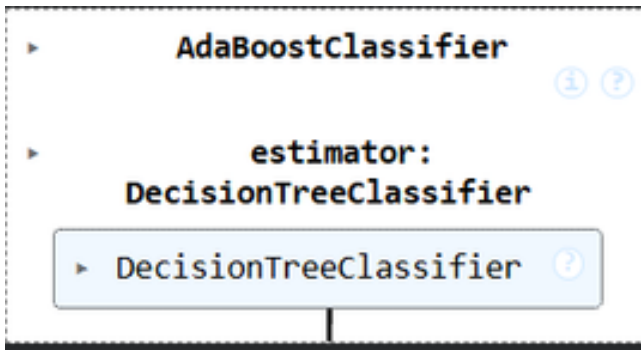
```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

Base Learner (Decision Stump)

```
base_model = DecisionTreeClassifier(max_depth=1)
```

AdaBoost Model

```
model = AdaBoostClassifier( estimator=base_model, n_estimators=50, learning_rate=1,  
random_state=42)model.fit(X_train, y_train)
```



Prediction

```
y_pred = model.predict(X_test)
```

Accuracy

```
print("Accuracy Score:")print(accuracy_score(y_test, y_pred))
```

Confusion Matrix Visualization

```
plt.figure(figsize=(6,4))sns.heatmap(cm, annot=True, fmt='d',  
cmap='Blues')plt.xlabel("Predicted")plt.ylabel("Actual")plt.title("Confusion Matrix -  
AdaBoost")plt.show()
```

Classification Report

```
print("Classification Report:")print(classification_report(y_test, y_pred))
```

Test Prediction

```
sample = X.iloc[0:1]prediction = model.predict(sample)print("Predicted Diabetes Class (0 = No, 1  
= Yes):", prediction[0])
```

Output:

Accuracy: 0.7792207792207793

Confusion Matrix:

[[80 19]

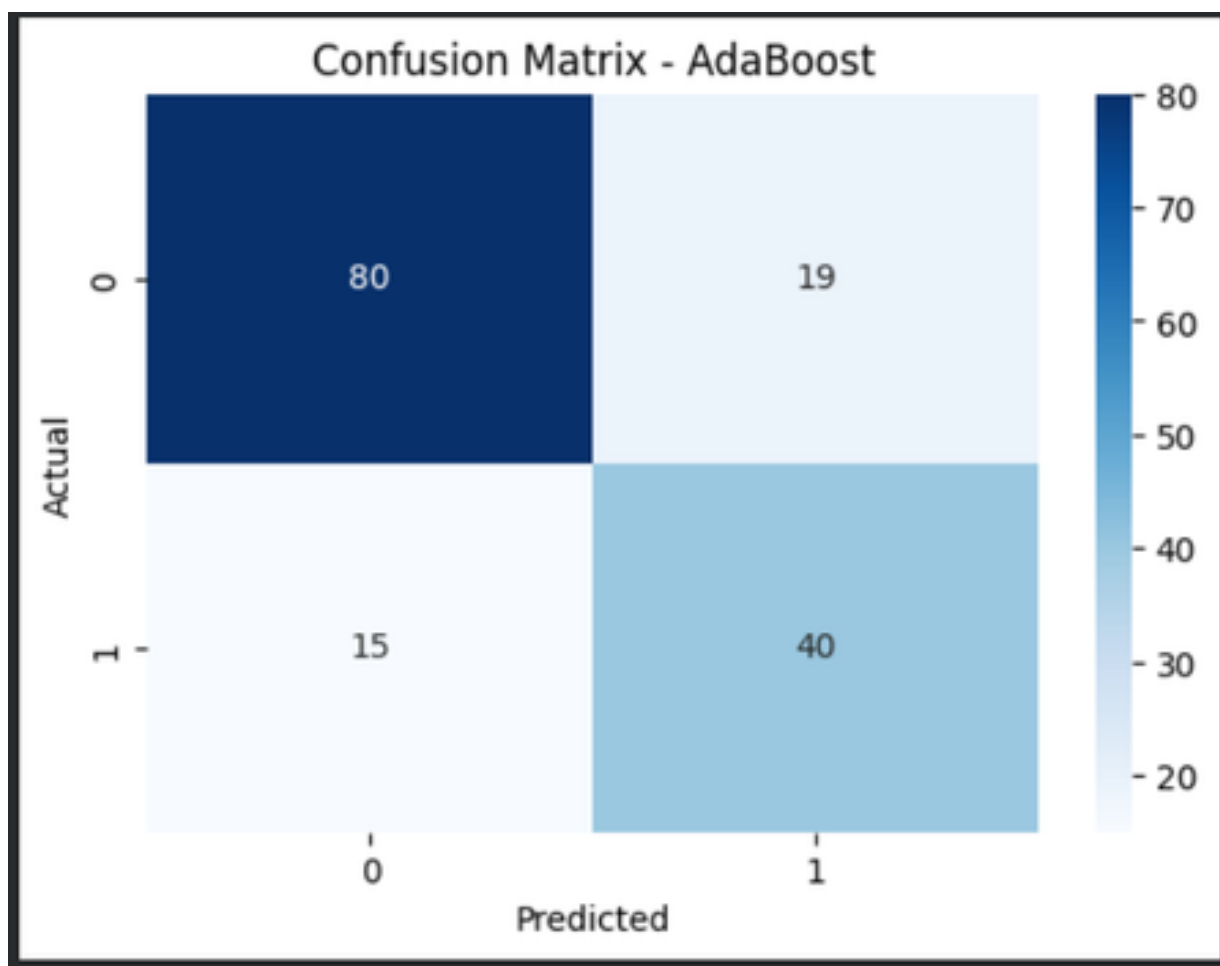
[15 40]]

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.81	0.82	99
1	0.68	0.73	0.70	55
accuracy			0.78	154
macro avg	0.76	0.77	0.76	154
weighted avg	0.78	0.78	0.78	154

Predicted Diabetes Class (0 = No, 1 = Yes): 1

Matrix visual output:



Result:

Thus, the AdaBoost algorithm was successfully implemented and the model accuracy was obtained.

Ex. No. : 09**Date:**

Register No.: 231701054 **Name: Srinivasan S**

A PYTHON PROGRAM TO IMPLEMENT K-NEAREST NEIGHBORS (KNN) CLASSIFIER

Aim:

To implement the K-Nearest Neighbors (KNN) algorithm on the diabetes dataset and classify whether a person has diabetes based on medical attributes.

Requirements:

- Jupyter Notebook / Google Colab
- Python
- diabetes.csv dataset
- Libraries: pandas, numpy, sklearn, matplotlib, seaborn

Algorithm:

K-Nearest Neighbors (KNN)

KNN is a supervised learning algorithm that classifies a data point based on the majority class of its nearest neighbors.

Process:

1. Load dataset
2. Split features and target
3. Normalize data (important for KNN)
4. Train KNN model
5. Predict output
6. Evaluate model

PROGRAM:

Upload Dataset

```
from google.colab import filesuploaded = files.upload()
```

Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("diabetes.csv")
print("Dataset Preview:")
print(df.head())
```

Split Features and Target

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Feature Scaling (VERY IMPORTANT for KNN)

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Apply KNN Model

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

Prediction

```
y_pred = model.predict(X_test)
```

Accuracy

```
print("Accuracy Score:")
print(accuracy_score(y_test, y_pred))
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

Confusion Matrix Visualization

```
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix - Diabetes Dataset")
plt.show()
```

Classification Report

```
print("Classification Report:")print(classification_report(y_test, y_pred))
```

Test Prediction

```
sample = X.iloc[0:1]prediction = model.predict(sample)print("Predicted Class (0 = No Diabetes, 1 = Diabetes):", prediction[0])
```

Output:

Accuracy Score:0.6948051948051948

Confusion Matrix:

```
[[79 20]
```

```
[27 28]]
```

Classification Report:

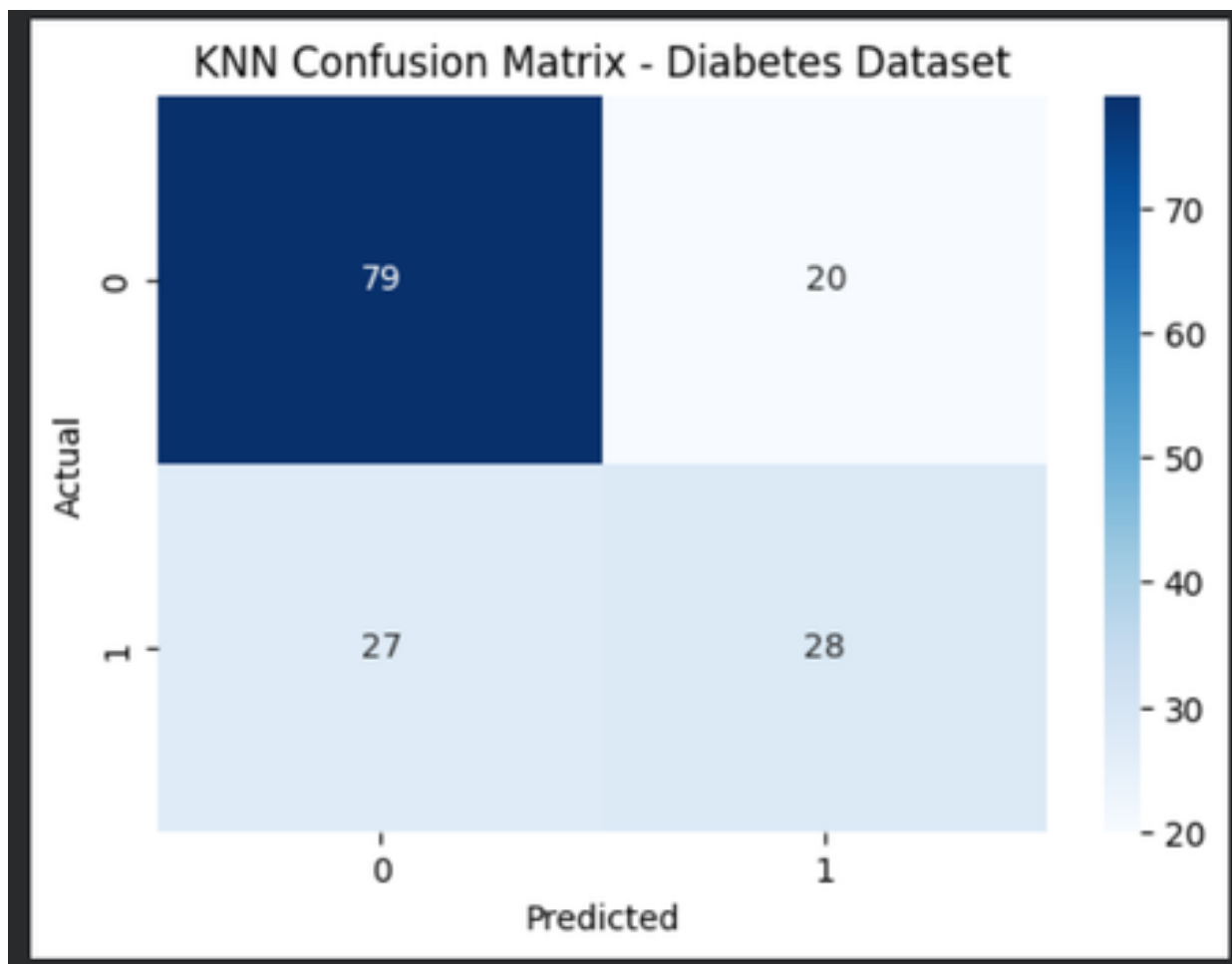
	precision	recall	f1-score	support
0	0.75	0.80	0.77	99
1	0.58	0.51	0.54	55
accuracy			0.69	154
macro avg	0.66	0.65	0.66	154
weighted avg	0.69	0.69	0.69	154

Predicted Class (0 = No Diabetes, 1 = Diabetes): 1

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but KNeighborsClassifier was fitted without feature names

```
warnings.warn(
```

Matrix visual:



Result:

Thus, the K-Nearest Neighbors (KNN) algorithm was successfully implemented using the diabetes dataset and used to classify whether a person has diabetes based on medical attributes.

A PYTHON PROGRAM TO IMPLEMENT K-MEANS CLUSTERING

Aim:

To implement K-Means clustering on the diabetes dataset and group similar data points into clusters based on feature similarity.

Requirements:

- Jupyter Notebook / Google Colab
- Python
- diabetes.csv dataset
- Libraries: pandas, sklearn, matplotlib

Algorithm:

K-Means Clustering

K-Means is an unsupervised learning algorithm that divides data into K clusters based on similarity.

Process:

1. Load dataset
2. Normalize data
3. Apply K-Means clustering
4. Assign cluster labels
5. Visualize clusters

PROGRAM:

Import Libraries

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("diabetes.csv")
print("Dataset Preview:")
print(df.head())
```

Prepare Data

```
X = df.drop("Outcome", axis=1)
```

Feature Scaling

```
scaler = StandardScaler()X_scaled = scaler.fit_transform(X)
```

Apply K-Means

```
kmeans = KMeans(n_clusters=2, random_state=42)kmeans.fit(X_scaled)
```

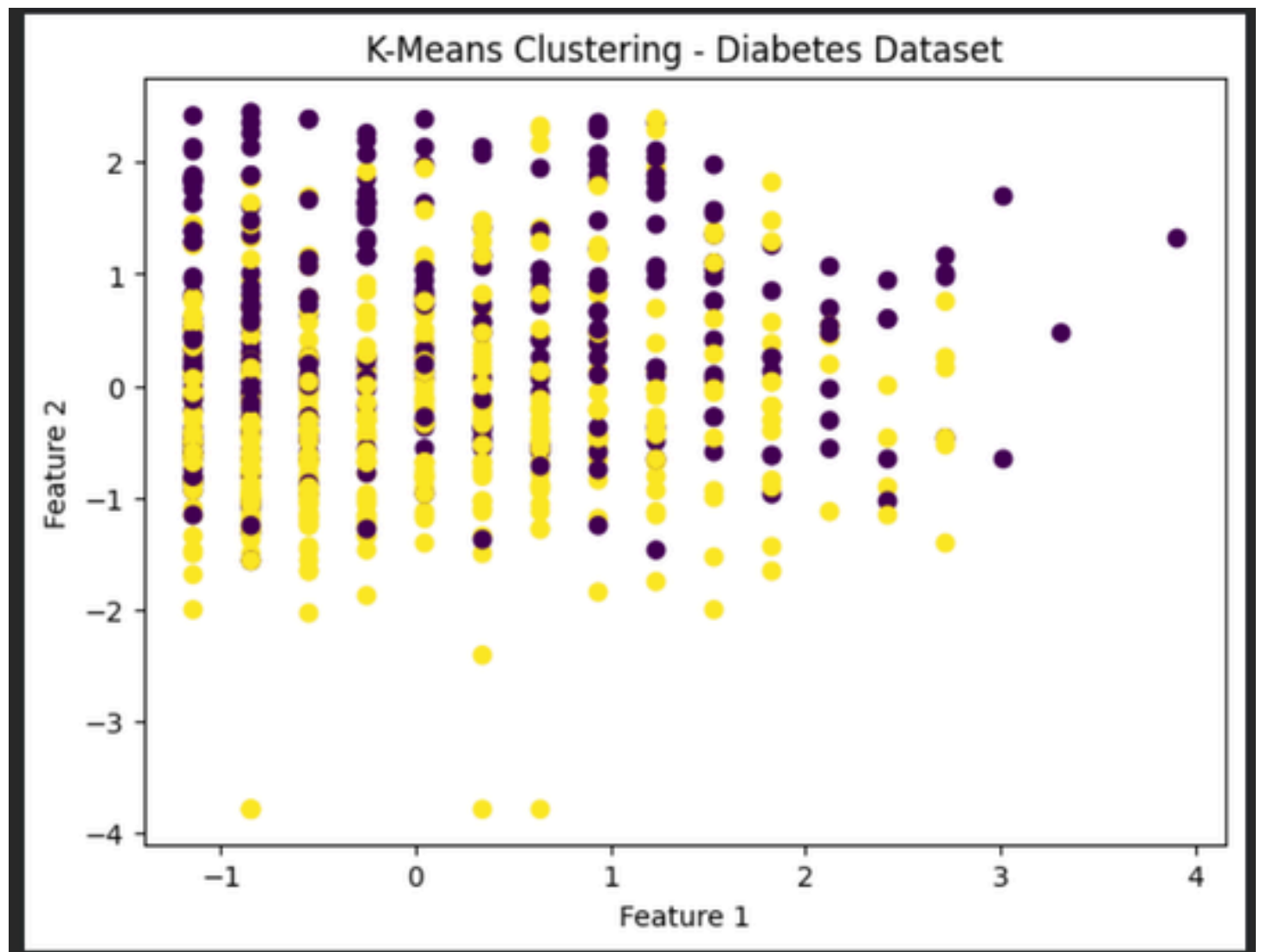
Cluster Labels

```
df["Cluster"] = kmeans.labels_print(df.head())
```

Visualization

```
plt.figure(figsize=(7,5))plt.scatter( X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_,  
cmap="viridis")plt.title("K-Means Clustering - Diabetes Dataset")plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")plt.show()
```

Output:



Result:

Thus, the K-Means clustering algorithm was successfully implemented using the diabetes dataset and grouped similar data points into clusters based on feature similarity.

Ex. No. : 10**Date:**

Register No.: 231701054 **Name: Srinivasan S**

A PYTHON PROGRAM TO IMPLEMENT DIMENSIONALITY REDUCTION USING PCA

Aim:

To implement Dimensionality Reduction using Principal Component Analysis (PCA) on the diabetes dataset and transform correlated features into a smaller set of uncorrelated principal components while preserving maximum variance.

Requirements:

- Jupyter Notebook / Google Colab
- Python
- diabetes.csv dataset
- Libraries: pandas, numpy, sklearn, matplotlib, seaborn

Algorithm:**Principal Component Analysis (PCA)**

PCA is a dimensionality reduction technique used to reduce the number of features in a dataset while preserving maximum variance. It converts correlated features into independent principal components.

Steps:

1. Load dataset
2. Standardize features
3. Analyze correlation
4. Apply PCA
5. Find explained variance
6. Reduce dimensions
7. Visualize results

PROGRAM:

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Load Dataset

```
data = pd.read_csv("diabetes.csv")
print("Dataset Preview:")
print(data.head())
```

Separate Features

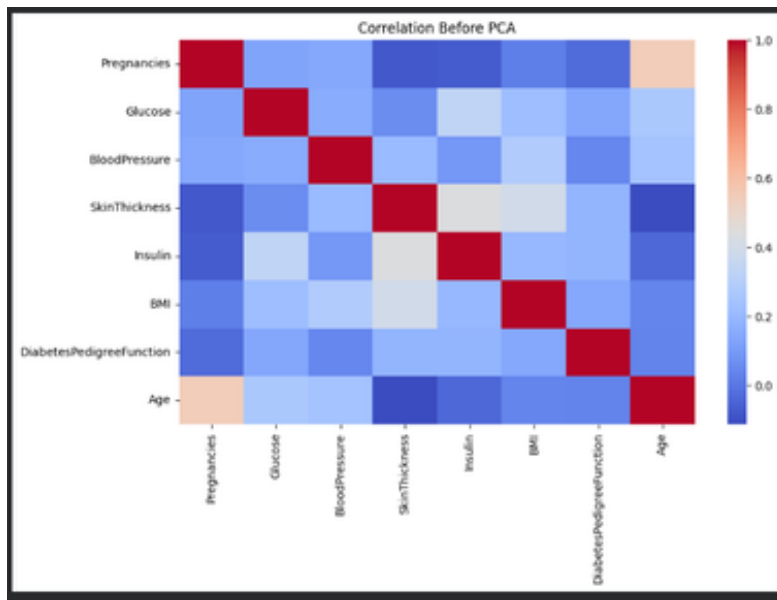
```
# Remove target column
X = data.drop("Outcome", axis=1)
```

Standardize Data

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled,
                        columns=X.columns)
```

Correlation Before PCA

```
plt.figure(figsize=(10,6))
sns.heatmap(X_scaled.corr(), cmap="coolwarm")
plt.title("Correlation Before PCA")
plt.show()
```



Apply PCA (All Components)

```
pca = PCA(pca.fit(X_scaled))
```

Explained Variance Ratio

```
explained_variance = pca.explained_variance_ratio_print("Explained Variance Ratio:")print(explained_variance)
```

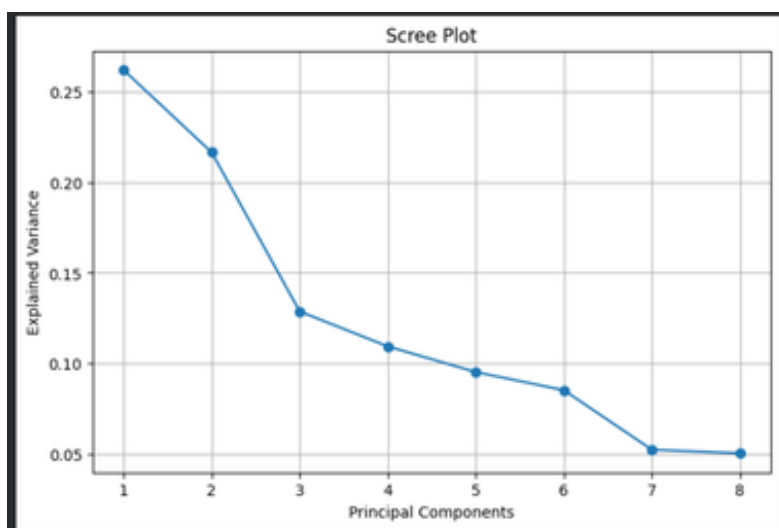
out:

Explained Variance Ratio:

```
[0.26179749 0.21640127 0.12870373 0.10944113 0.09529305 0.08532855
 0.05247702 0.05055776]
```

Scree Plot

```
plt.figure(figsize=(8,5))plt.plot(range(1, len(explained_variance)+1), explained_variance,
marker='o')plt.title("Scree Plot")plt.xlabel("Principal Components")plt.ylabel("Explained Variance")plt.grid()plt.show()
```



Apply PCA (2 Components)

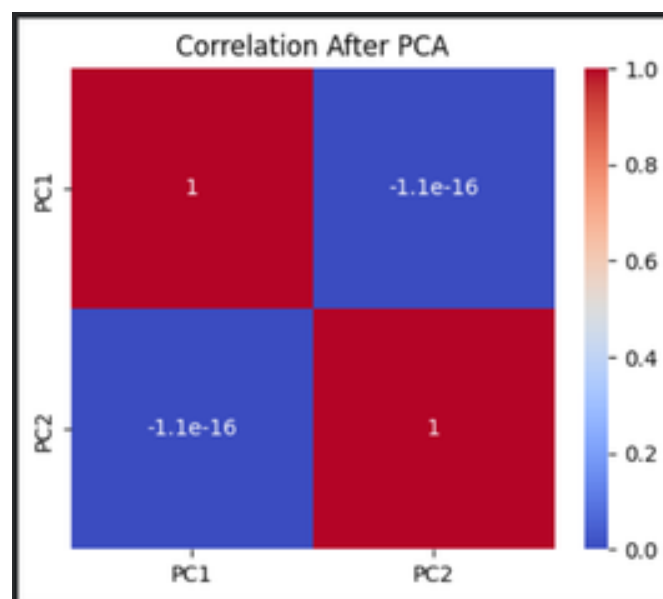
```
pca_final = PCA(n_components=2)X_pca = pca_final.fit_transform(X_scaled)X_pca =  
pd.DataFrame(X_pca, columns=["PC1", "PC2"])print(X_pca.head())
```

OUT:

	PC1	PC2
0	1.068503	1.234895
1	-1.121683	-0.733852
2	-0.396477	1.595876
3	-1.115781	-1.271241
4	2.359334	-2.184819

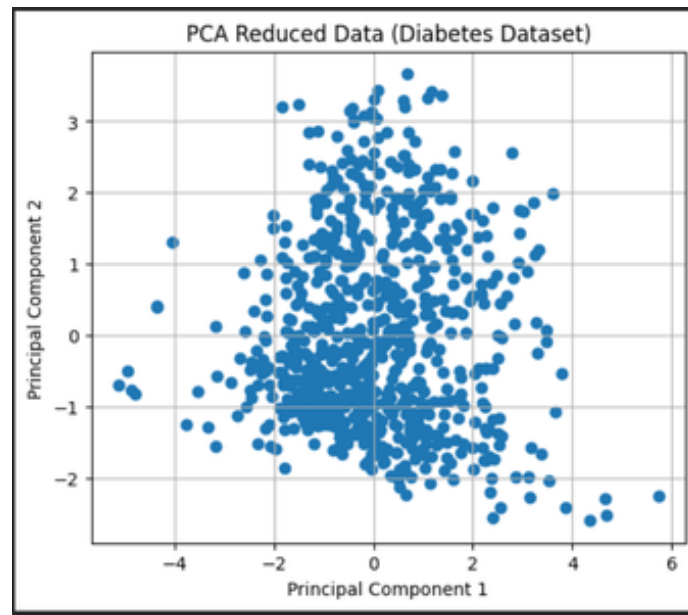
Correlation After PCA

```
plt.figure(figsize=(5,4))sns.heatmap(X_pca.corr(), annot=True,  
cmap="coolwarm")plt.title("Correlation After PCA")plt.show()
```



PCA Visualization

```
plt.figure(figsize=(6,5))plt.scatter(X_pca["PC1"], X_pca["PC2"])plt.xlabel("Principal Component  
1")plt.ylabel("Principal Component 2")plt.title("PCA Reduced Data (Diabetes  
Dataset)")plt.grid()plt.show()
```



RESULT:

Thus, Principal Component Analysis (PCA) was successfully implemented on the diabetes dataset and reduced the dimensionality of the data while preserving maximum variance.